

Metropolia University of Applied Sciences

Software Engineering Project Final Report

Project: Fliply

Course	Software Engineering Project
Application Type	JavaFX Desktop Learning Application
Repository	https://github.com/nguyngc/fliply
Date	April 2026
Team	Ngoc Nguyen, Thanh Nguyen, Nhut Vo, Hoang Vu

Table of Contents

1. Introduction	3
2. Customer Requirements	3
3. Development Methods	4
4. System Design and Architecture	5
5. Implementation	9
6. Testing	11
7. Summary	13
References	14
Appendices	15

1. Introduction

Fliply is a JavaFX desktop application designed to support students and teachers in managing learning materials through flashcards, quizzes, and classroom-based content. The main idea of the project is to create one lightweight learning platform where teachers can publish study materials and students can practise them in a more structured way.

The application was developed as part of the Software Production Project course. The system supports common learning features such as user authentication, flashcard management, quiz generation, quiz result tracking, classroom management, account management, and multilingual user interface support. According to the project README, Fliply supports English, Vietnamese, Lao, Korean, Finnish, and Arabic user interfaces.

The project was developed over eight 2-week sprints, with a total development time of 16 weeks. The main objective of the project was not only to build a working application, but also to practise software production methods such as Agile/Scrum, requirement analysis, system design, implementation, testing, CI/CD, documentation, and teamwork.

2. Customer Requirements

The customer requirement was to build a simple but structured learning platform for both students and teachers. The product vision describes Fliply as a system where teachers can publish study content and students can practise it using flashcards, quizzes, and class-based learning materials.

The main customer requirements were:

- The system must support both student and teacher roles.
- Users must be able to register, log in, and manage their accounts.
- Teachers must be able to create and manage classes.
- Teachers must be able to create flashcard sets and learning content.
- Students must be able to study flashcards and take quizzes.
- The system should track quiz results.
- The user interface should support multiple languages.
- The system should be testable, maintainable, and easy to run.

The system was considered successful when the main workflows worked correctly, automated tests passed, localization was implemented, documentation was completed, and the application could be built and run reproducibly.

3. Development Methods

The project was developed using an Agile/Scrum approach. The whole project was divided into eight sprints, and each sprint focused on a specific part of the system. The sprint structure helped the team move from planning and requirements to UI implementation, Docker setup, localization, testing, and final documentation.

Sprint	Main Focus
Sprint 1	Project planning, product vision, scope, backlog, and risks
Sprint 2	Functional requirements, use cases, and database design
Sprint 3	JavaFX UI implementation and CI pipeline setup
Sprint 4	Docker containerization and reproducible runtime setup
Sprint 5	UI localization and language support
Sprint 6	Database localization and acceptance testing preparation
Sprint 7	Quality assurance, static analysis, and test stabilization
Sprint 8	Final documentation and repository finalization

Table 1. Sprint structure and main focus areas

The team used sprint planning and review documents to track progress. GitHub was used for source code management, while Trello or project tracking tools were used to organize tasks and follow the workflow. This helped the team divide work between members and keep the project organized during the course.

4. System Design and Architecture

Fliply follows a desktop application structure based on JavaFX. The system uses FXML files for views, controllers for handling user actions, services for business logic, and JPA/Hibernate entities for database communication. This makes the application easier to maintain because the UI, logic, and data layers are separated.

The main architectural parts are:

- View layer: JavaFX FXML screens such as Welcome, Login, Register, Home, Flashcard, Quiz, and Account screens.
- Controller layer: JavaFX controllers that handle button actions, form input, navigation, and user interaction.
- Service layer: business logic for users, flashcards, quizzes, classes, and localization.
- Data layer: JPA/Hibernate entities and DAO/repository classes connected to MariaDB.
- Database: MariaDB database storing users, classes, flashcards, quizzes, and localized content.
- Localization: Java ResourceBundle files and database fields for multilingual UI/content.
- Build and deployment: Maven, Docker, Docker Compose, and Jenkins.

The README lists several diagrams used in the project documentation, including Use Case Diagram, ER Diagram, Relational Schema, Activity Diagrams, Class Diagram, Sequence Diagram, Deployment Diagram, and User Journey Map.

4.1 ER Diagram

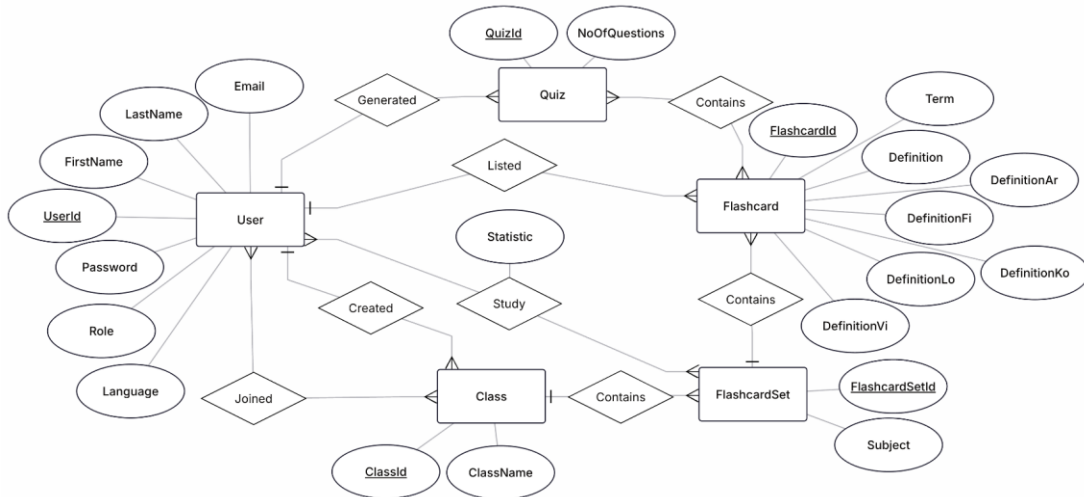


Figure 1. ER diagram for the Fliply database structure

The ER diagram describes how the main database entities are connected. The main entities include User, Class, FlashcardSet, Flashcard, Quiz, and quiz-related result data. The database was designed in Sprint 2 and implemented using MariaDB with JPA/Hibernate mapping.

4.2 Use Case Diagram

FlashCard System – Use Case Diagram

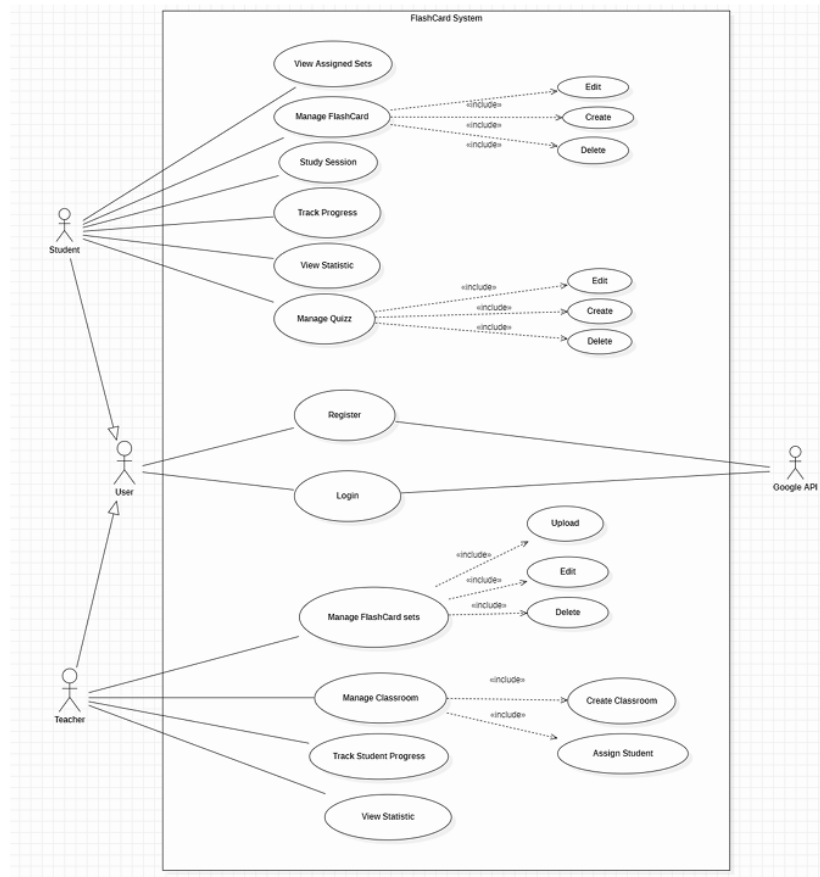


Figure 2. Use case diagram showing student and teacher interactions

The use case diagram shows the main actions that students and teachers can perform. For example, students can register, log in, study flashcards, take quizzes, and view learning content. Teachers can manage classes, create flashcard sets, and prepare learning materials.

4.3 Activity Diagrams

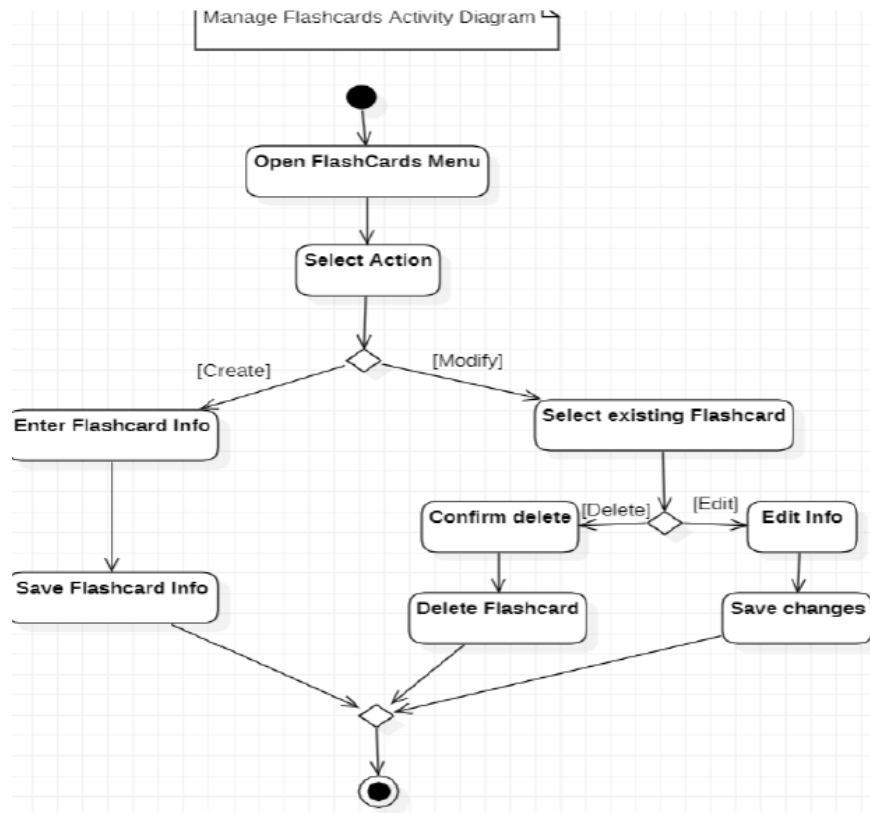


Figure 3. Activity diagram for managing flashcards

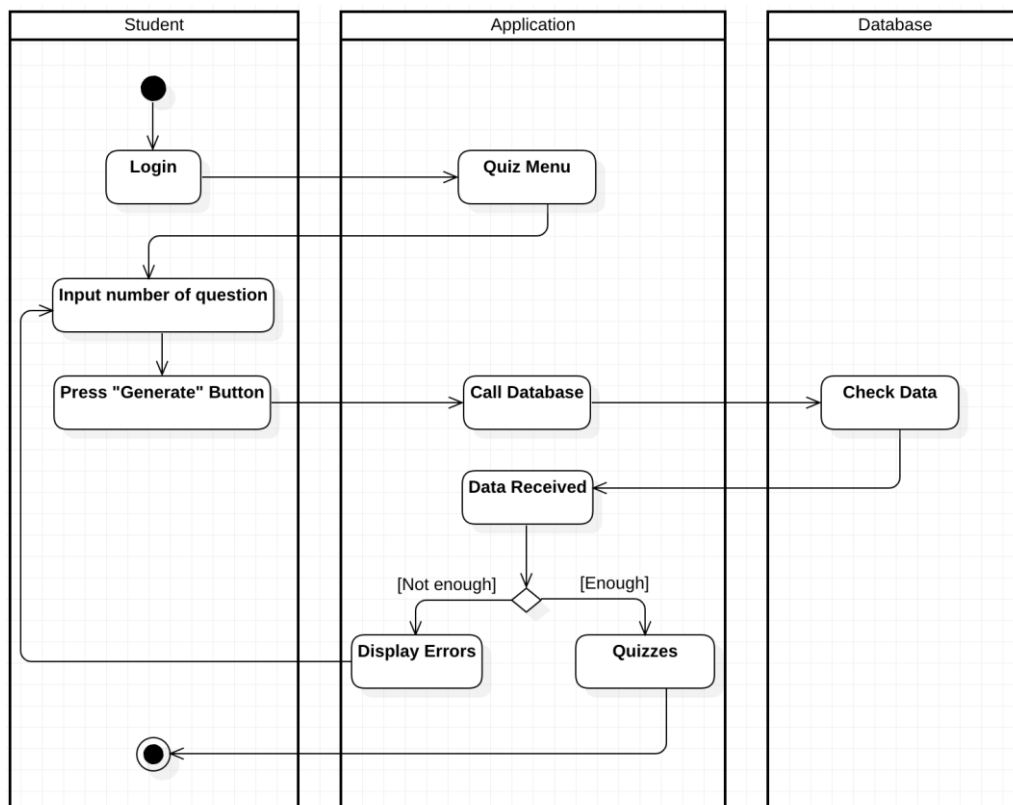


Figure 4. Activity diagram for quiz generation

The activity diagrams explain the flow of important functions. For example, the quiz generation process shows how a user starts a quiz, how the system selects questions, and how the result is produced. The flashcard management activity diagram shows the flow for creating, editing, or deleting flashcard content.

4.4 Sequence Diagram

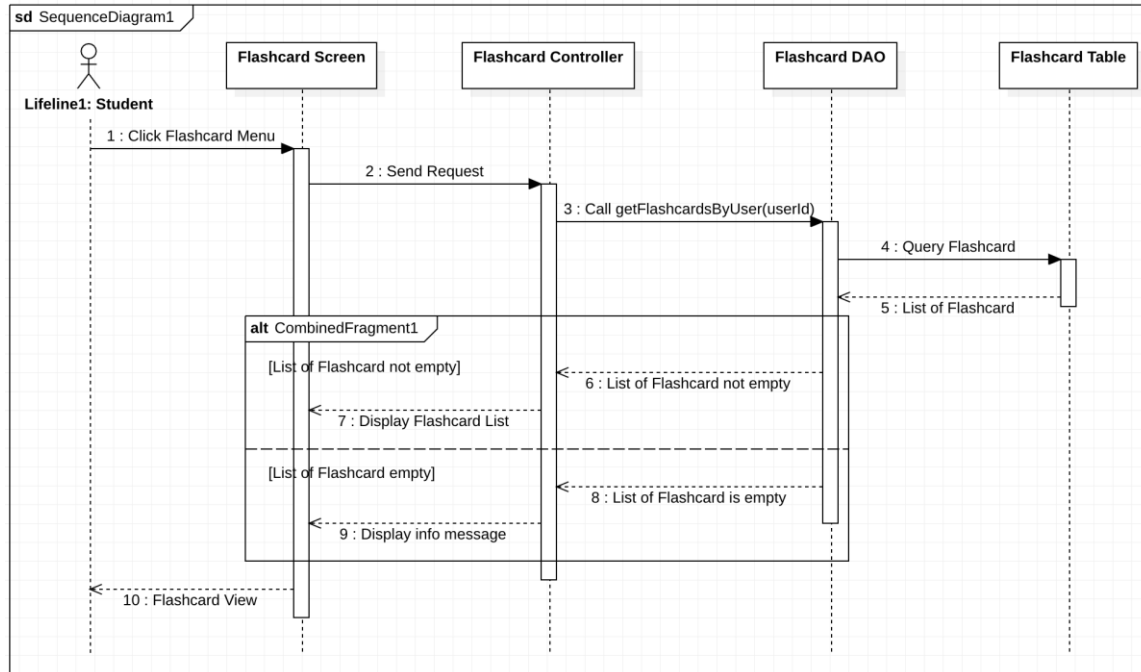


Figure 5. Sequence diagram for a flashcard workflow

The sequence diagram shows how objects interact during one main workflow. For example, when a user creates or studies flashcards, the controller receives the user action, calls the service layer, and the service communicates with the database through JPA/Hibernate.

5. Implementation

5.1 Hardware

No special hardware was required for this project. The application can be run on a normal development computer that supports Java 21, Maven, MariaDB, and JavaFX.

5.2 Software

The main technologies used in Fliply were:

Technology	Purpose
Java 21	Main programming language
JavaFX	Desktop GUI framework
FXML	UI layout structure
Maven	Build and dependency management
MariaDB	Database
JPA/Hibernate	Object-relational mapping
JUnit 5	Unit testing
Mockito	Mocking for tests
JaCoCo	Test coverage report
Docker	Containerization
Docker Compose	Running app and database together
Jenkins	CI pipeline
SonarQube	Static code analysis

Table 2. Main software technologies used in Fliply

5.3 Main Features Implemented

- User registration and login
- Role-based student and teacher screens
- Flashcard set management
- Flashcard creation and study mode
- Quiz generation
- Quiz result tracking
- Class management
- Account management

- Multilingual UI
- Localized flashcard definitions
- Docker-based setup
- Automated testing and code coverage

5.4 Localization Implementation

One important part of the implementation was localization. Fliply supports six languages: English, Vietnamese, Lao, Korean, Finnish, and Arabic. The UI localization was implemented using Java ResourceBundle files. The application can reload screens based on the selected language, and the selected language can be saved as a user preference.

In Sprint 6, localization was extended to the database level. Flashcard definitions were expanded with language-specific fields, and fallback logic was added so that English content can be shown if the selected language is not available.

5.5 Running the Application

The application can be run locally with Maven:

```
mvn clean install
mvn javafx:run
```

It can also be run using Docker Compose:

```
docker compose up --build
```

When running with Docker, GUI forwarding or an X server may be needed because Fliply is a JavaFX desktop application.

6. Testing

Testing was done throughout the project, but the main quality assurance work was completed during Sprint 7. The testing process included unit testing, integration testing, functional testing, manual test cases, SonarQube analysis, and usability evaluation.

6.1 Unit Testing

Unit testing was implemented using JUnit 5. Mockito was used where mocking was needed. The purpose of unit testing was to check individual parts of the system, such as services, validation logic, and smaller business logic functions.

The project can run unit tests using:

```
mvn test
```

The coverage report can be generated using:

```
mvn verify
```

After running verification, the JaCoCo report can be opened from:

```
target/site/jacoco/index.html
```

6.2 Integration Testing

Integration testing focused on checking that different layers of the system worked together correctly. For example, controllers, services, JPA/Hibernate entities, and the MariaDB database needed to work together in workflows such as login, flashcard creation, class management, and quiz handling.

Some integration-related checks were also connected to Docker and database setup, because the team needed to make sure the system could run with the database service and seed data.

6.3 Functional Testing

Functional testing was done manually using test cases. The main goal was to check whether user-facing features worked correctly from the user's point of view. Example tested workflows included:

- Registering a new user
- Logging in with valid and invalid credentials
- Creating and managing flashcards
- Taking quizzes
- Managing account information
- Switching language
- Checking validation messages
- Testing teacher and student workflows

According to the README, Sprint 7 included a comprehensive functional test suite, more than 60 manual test cases, SonarQube analysis, heuristic evaluation, and bug fixing. It also mentions bug fixes related to empty input validation, flashcard deletion confirmation, quiz question count validation, and account form validation.

6.4 Quality Assurance and Static Analysis

SonarQube was used to analyze code quality and identify possible issues. JaCoCo was used for code coverage. The project also included usability evaluation using Nielsen's heuristics, which helped the team find UI/UX problems and improve user interaction.

Some issues found during testing were fixed before final delivery, especially validation errors and missing confirmation messages.

7. Summary

The Fliply project successfully delivered a JavaFX desktop learning application for students and teachers. The system includes the main planned features such as authentication, flashcard management, quiz generation, classroom management, account management, and multilingual support.

During the project, the team practised the full software production process, including planning, requirement analysis, system design, implementation, testing, CI/CD, Docker setup, and documentation. Agile/Scrum helped the team divide the work into smaller sprint goals and improve the system step by step.

The most successful parts of the project were the clear role-based workflows, multilingual UI support, database localization, Docker setup, and final documentation structure. The testing phase also helped improve the system by fixing validation problems and improving reliability.

Some challenges included managing database integration, keeping localization consistent across many languages, and making sure the JavaFX application could run properly in different environments. Since Fliply is a desktop GUI application, Docker execution required extra attention because GUI forwarding may be needed.

Overall, the project gave the team practical experience in building a complete software product from planning to final delivery.

References

Fliply GitHub Repository: <https://github.com/nguyngc/fliply>

Fliply README.md, project overview, product vision, sprint structure, documentation hub, testing instructions, and repository structure.

Project documents referenced in the repository: Product Vision, Project Plan, Product Backlog, Sprint Planning Reports, Sprint Review Reports, ER Diagram, Use Case Diagram, Activity Diagrams, Sequence Diagram, Class Diagram, Deployment Diagram, Test Plan, Test Case Excel, Heuristic Evaluation Report, Database Localization Report, and Localization Workbook.

Appendices

Appendix A: Diagrams

- ER Diagram
- Use Case Diagram
- Activity Diagram – Manage Flashcards
- Activity Diagram – Generate Quiz
- Sequence Diagram

The main diagrams are included in Section 4 to support the architecture explanation. Additional diagrams such as Deployment Diagram or User Journey Map can be added here if the team wants to provide more supplementary material.

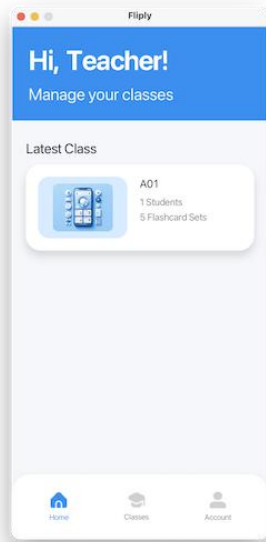
Appendix B: Screenshots



Welcome screen



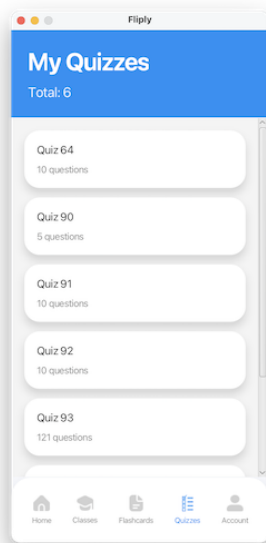
Student home screen



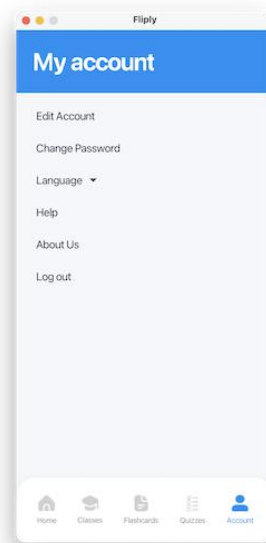
Teacher home screen



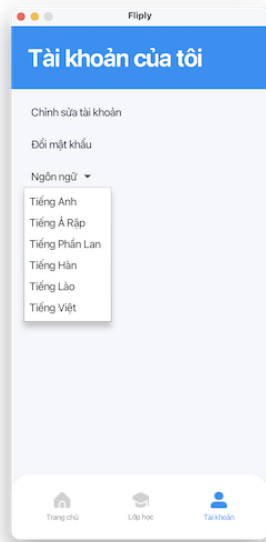
Flashcard screen



Quiz screen



Account screen



Vietnamese localization screen



Arabic RTL language screen

Figure 6. Main application screenshots

Appendix C: Testing Evidence

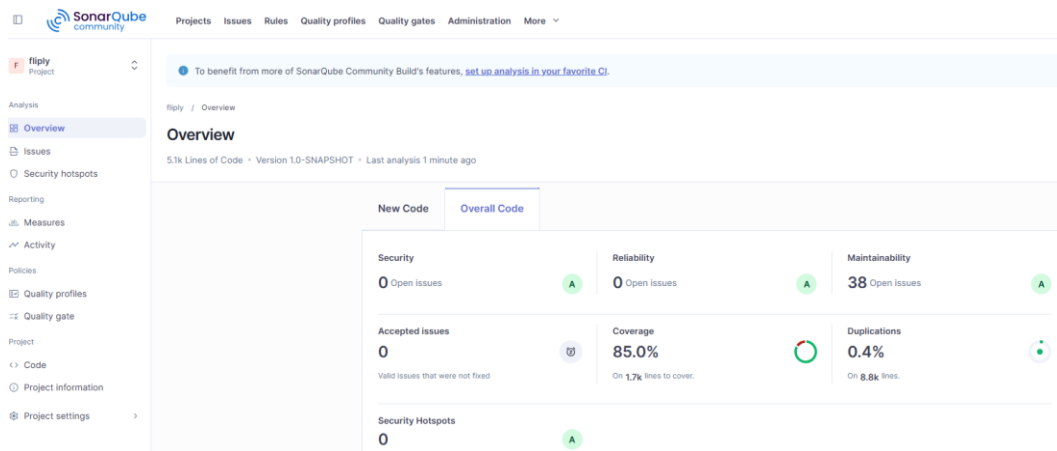


Figure 7. SonarQube report screenshot

```
[INFO] Results:
[INFO]
[WARNING] Tests run: 273, Failures: 0, Errors: 0, Skipped: 6
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ fliply ---
[INFO] Building jar: C:\Users\hiryu\IdeaProjects\fliply\target\fliply-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- jacoco:0.8.14:report (report) @ fliply ---
[INFO] Loading execution data file C:\Users\hiryu\IdeaProjects\fliply\target\jacoco.exec
[INFO] Analyzed bundle 'fliply' with 72 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 31.706 s
[INFO] Finished at: 2026-04-14T13:05:00+03:00
[INFO]
```

Figure 8. Unit test / JaCoCo evidence screenshot

Appendix D: Tools

- GitHub for source code management
- Trello / GitHub project board for task tracking
- IntelliJ IDEA for development
- StarUML for diagrams
- Maven for build management
- MariaDB for database
- Docker and Docker Compose for container setup
- Jenkins for CI pipeline
- JUnit 5 and Mockito for testing
- JaCoCo for code coverage
- SonarQube for static code analysis
- Excel/PDF documents for test cases and reports

Appendix E: Project Links

GitHub Repository: <https://github.com/nguyngc/fliply>

Trello Link: https://trello.com/w/sep1_group3